

《数据库系统原理》实验报告（实验五）

题目：实验五：miniOB 入门实验

学号	2351520	姓名	毛星博	日期	2025.5.8
----	---------	----	-----	----	----------

实验环境：Docker OceanBase-miniob

实验步骤及结果截图：

1.在 Docker 中建立 miniob 环境

```
# cd build
/bin/sh: 1: cd: can't cd to build
# cd miniob
# cd build
# ./bin/observer -s miniob.sock -f ../etc/observer.ini &
#
Welcome to the OceanBase database implementation course.

Copyright (c) 2021 OceanBase and/or its affiliates.

Learn more about OceanBase at https://github.com/oceanbase/oceanbase
Learn more about MiniOB at https://github.com/oceanbase/miniob

Successfully load ../etc/observer.ini
./bin/obclient -s miniob.sock

Welcome to the OceanBase database implementation course.

Copyright (c) 2021 OceanBase and/or its affiliates.

Learn more about OceanBase at https://github.com/oceanbase/oceanbase
Learn more about MiniOB at https://github.com/oceanbase/miniob
```

2.创建 Student 表

```
miniob > create table Student (
  No int,
  Name char(10),
  Grade float,
  Credit float
);
SUCCESS
```

3.插入数据

```
miniob > insert into Student values (2350001, '张三', 4.8, 120.0);
SUCCESS
miniob > insert into Student values (2350002, '李四', 4.7, 124.5);
SUCCESS
miniob > insert into Student values (2350003, '王五', 4.2, 99.5);
SUCCESS
miniob > insert into Student values (2351520, '毛星博', 5.0, 150)
FAILURE
miniob > insert into Student values (2351520, '毛星博', 5.0, 150.0)
SUCCESS _
```

4.使用 select 语句展示学号，绩点，学分

```
miniob > select No,Grade,Credit from Student;
No | Grade | Credit
2350001 | 4.8 | 120
2350002 | 4.7 | 124.5
2350003 | 4.2 | 99.5
2351520 | 5 | 150
```

5. 尝试修改指定行的绩点或学分

```
miniob > update Student set Grade=4.82 where No=2350001;
FAILURE
miniob > update Student set Grade = 4.65 where No = 2350002;
FAILURE
miniob > update Student set Credit = 103.5 where No = 2350003;
FAILURE
```

修改不成功。原因：

```
UpdateStmt::UpdateStmt(Table *table, Value *values, int value_amount)
| | : table_(table), values_(values), value_amount_(value_amount)
| | {}
```

查看源代码后发现，update 函数还没有实现

6. 删除张三的记录

```
miniob > delete from Student where Name='张三';
SUCCESS
miniob > select * from Student;
No | Name | Grade | Credit
2350002 | 李四 | 4.7 | 124.5
2350003 | 王五 | 4.2 | 99.5
2351520 | 毛星博 | 5 | 150
```

7. 使用 select 语句展示我的学号

```
miniob > select No from Student where Name = '毛星博';
No
2351520 _
```

8. 对 miniob 源码进行阅读，主要选取一个功能（如 create table、insert、delete 等）进行分析理解，做简要报告（不超过两页）

见第 3,4 页

出现的问题：

在插入数据 insert into Student values (2350001, '张三', 4.8, 120) 时，总是显示失败

```
miniob > insert into Student values (2350001, '张三', 4.8, 120);
FAILURE
```

解决方案：

将 120 修改为 120.0，insert into Student values (2350001, '张三', 4.8, 120.0) 即可成功

```
miniob > insert into Student values (2350001, '张三', 4.8, 120.0);
SUCCESS
```

MiniOB——create table 建表功能 源码分析报告

1. SQL 解析阶段

当用户输入 create table 语句时, MiniOB 首先通过词法分析器 (lex) 和语法分析器 (yacc) 进行处理。

语法规则定义在 yacc_sql 中。

create_table_stmt 规则用于解析整个建表语句

而 attr_def_list 和 attr_def 用于解析列定义

解析的结果存储在 CreateTableSqlNode 和 AttrInfoSqlNode 结构体中

```
// @brief 描述一个属性
struct AttrInfoSqlNode
// @brief 描述一个 create table 语句
struct CreateTableSqlNode
```

CreateTableSqlNode 包含:

- relation_name: string, 表名
- attr_infos: vector<AttrInfoSqlNode>, 存储所有列的定义。
- primary_keys: vector<string>, 存储主键列名。
- storage_format: 存储格式字符串。

AttrInfoSqlNode 包含:

- type: AttrType 枚举, 列的数据类型
- name: string, 列名
- length: size_t, 列的长度

2. 语句对象构建阶段

解析得到的 CreateTableSqlNode 随后被 Stmt::create_stmt 方法转换为一个更结构化的语句对象 CreateTableStmt。

```
Stmt::create_stmt(Db *db, ParsedSqlNode &sql_node, Stmt *&stmt)
{
    switch (sql_node.flag) {
        case SCF_CREATE_TABLE: {
            return CreateTableStmt::create(db, sql_node.create_table, stmt);
        }
    }
}
```

CreateTableStmt::create 负责接收 CreateTableSqlNode, 并构造一个 CreateTableStmt 对象。

CreateTableStmt 持有表名、attr_infos 向量、主键列表以及解析后的 StorageFormat 枚举。

3. 执行与调度

CreateTableStmt 对象创建后, 持有 CreateTableStmt 对象的 SQLStageEvent 被创建, 会被传递给 SQL 执行引擎进行处理。

CreateTableExecutor::execute 方法会从 SQLStageEvent 中获取 CreateTableStmt, 并从当前 Session 获取 Db 对象, 然后调用 Db::create_table。

```
RC CreateTableExecutor::execute(SQLStageEvent *sql_event)
{
    Stmt *stmt = sql_event->stmt();
    Session *session = sql_event->session_event()->session();
```

```

ASSERT(stmt->type() == StmtType::CREATE_TABLE,
    "create table executor can not run this command: %d",
    static_cast<int>(stmt->type()));
CreateTableStmt *create_table_stmt = static_cast<CreateTableStmt *>(stmt);
const char *table_name = create_table_stmt->table_name().c_str();
RC rc = session->get_current_db()->create_table(table_name, create_table_stmt->attr_infos(),
create_table_stmt->primary_keys(), create_table_stmt->storage_format());
return rc;
}

```

4.数据库层 Db 类

Db 类代表一个数据库实例，并管理其中的表。

Db::create_table 方法是处理建表请求的一个关键入口点，它会检查表是否存在，分配表 ID，并调用 Table 对象的创建逻辑。

```

// @brief 一个 DB 实例负责管理一批表
class Db
{
    RC create_table(const char *table_name, span<const AttrInfoSqlNode> attributes, const vector<string> &primary_keys,
        const StorageFormat storage_format = StorageFormat::ROW_FORMAT);
    ...
};

```

5.表对象与元数据处理 (Table 和 TableMeta 类)

Table 类代表一个具体的表。

Table::create 方法是实际执行表创建、元数据初始化和存储文件、储存引擎准备的方法。

```

/* 创建一个表
 * @param path 元数据保存的文件(完整路径)
 * @param name 表名
 * @param base_dir 表数据存放的路径
 * @param attributes 字段
 */
RC create(Db *db, int32_t table_id, const char *path, const char *name, const char *base_dir,
    span<const AttrInfoSqlNode> attributes, const vector<string> &primary_keys, StorageFormat storage_format,
    StorageEngine storage_engine);

```

此方法内部会调用 TableMeta::init 来根据 AttrInfoSqlNode 列表初始化表元数据 TableMeta，并为每一列创建 FieldMeta 对象

FieldMeta 存储了列的详细属性，如类型、长度、名称以及在物理记录中的偏移量。

```

FieldMeta::FieldMeta(const char *name, AttrType attr_type, int attr_offset, int attr_len, bool visible, int field_id)
{
    [[maybe_unused]] RC rc = this->init(name, attr_type, attr_offset, attr_len, visible, field_id);
    ASSERT(rc == RC::SUCCESS, "failed to init field meta. rc=%s", strrc(rc));
}

```

6.最终，各层函数将执行结果 RC 逐级返回，客户端会收到一个成功或失败的响应。